Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session	**2025-26 (ODD)**	**Course Name**	**Operating System Lab**
Semester	**5**	**Course Code**	**23IOT1504**
Roll No	**64**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**04**
Course Outcome	1. Understand Computer System Configuration and Simulate system resources efficiently using Linux Commands (CO1) 2. Analyse operating system functionalities utilizing system calls, thread programming and process scheduling algorithms (CO2) 3. Apply Synchronization primitives to implement a Deadlock-free solution(CO3) 4. Simulate Disk scheduling, Memory allocation, File allocation, page replacement algorithms (CO4)
Aim	Write a simulator program CPU scheduling algorithms. A. Non Pre-emptive CPU Scheduling Algorithm(Any one) B. Pre-emptive CPU Scheduling Algorithm
Theory	**Non-Preemptive Scheduling** In non-preemptive scheduling, once a process begins execution on the CPU, it continues until it either finishes or voluntarily enters a waiting state. The operating system does not forcefully stop or replace it. **Examples**: First Come First Serve (FCFS), Shortest Job First (SJF). **Advantages**: Very little scheduling overhead. Requires fewer system resources. **Disadvantages**: A single long or faulty process may block others indefinitely. Response time is generally poor since time-sharing methods like Round Robin cannot be used. **Preemptive Scheduling** In preemptive scheduling, the operating system can pause a running process and allocate the CPU to another process, usually based on priority or time-sharing. A process may be shifted from the Running state back to Ready before completing. **Examples**: Round Robin, Shortest Remaining Time First (SRTF), Priority Scheduling (preemptive). **Advantages:** Prevents CPU monopolization by a single process.

	Provides better response times in interactive and multi-user systems. **Disadvantages:** More complex to design and implement. Extra overhead due to frequent context switching.
Procedure and Execution	**Code:** **A. Non-Preemptive CPU Scheduling (FCFS).**

```
// Practical_4_OS - Non-Preemptive Scheduling (FCFS)
// By Sheikh Wasimuddin
#include <stdio.h>
int main() {
    int n, i;
    int at[100], bt[100], wt[100], tat[100], ct[100];
    float total_wt = 0, total_tat = 0;

    printf("Enter  Total Number Of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter The Arrival Time for P%d: ", i + 1);
        scanf("%d", &at[i]);
        printf("Enter The  Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort
    for (i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                int t;
                t = at[j];
                at[j] = at[j + 1];
                at[j + 1] = t;
                t = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = t;
            }
        }
    }

    ct[0] = at[0] + bt[0];
    tat[0] = ct[0] - at[0];
    wt[0] = tat[0] - bt[0];
```

```
    for (i = 1; i < n; i++) {
        if (ct[i - 1] < at[i]) {
            ct[i] = at[i] + bt[i];
        } else {
            ct[i] = ct[i - 1] + bt[i];
        }
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    printf("\n--- FCFS Scheduling ---\n");
    printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++) {
        total_wt += wt[i];
        total_tat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i],
wt[i]);
    }

    printf("\n The Average WT = %.2f ms", total_wt / n);
    printf("\n The Average TAT = %.2f ms\n", total_tat / n);

    return 0;
}
```

B. Preemptive CPU Scheduling (SRTF – Shortest Remaining Time First)

```
// Practical_4_OS - Preemptive Scheduling (SRTF)
// By Sheikh Wasimuddin

#include <stdio.h>

int main() {
    int n, i, time = 0, completed = 0;
    int at[100], bt[100], rt[100];
    int wt[100] = {0}, tat[100];
    float total_wt = 0, total_tat = 0;

    printf("Enter Total Number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter The Arrival Time for P%d: ", i + 1);
```

```
        scanf("%d", &at[i]);
        printf("Enter The Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    while (completed != n) {
        int shortest = -1, min_time = 1e9;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0 && rt[i] < min_time) {
                min_time = rt[i];
                shortest = i;
            }
        }
        if (shortest == -1) {
            time++;
            continue;
        }
        rt[shortest]--;
        time++;
        if (rt[shortest] == 0) {
            completed++;
            int finish_time = time;
            wt[shortest] = finish_time - bt[shortest] - at[shortest];
            if (wt[shortest] < 0) wt[shortest] = 0;
        }
    }
    for (i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
    }
    printf("\n--- SRTF Scheduling ---\n");
    printf("Process\tAT\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        total_wt += wt[i];
        total_tat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
    }

    printf("\n The Average WT = %.2f ms", total_wt / n);
    printf("\n The Average TAT = %.2f ms\n", total_tat / n);

    return 0;
}
```

Output:

A. Non-Preemptive CPU Scheduling (FCFS).

main.c

```c
// Practical_4_OS - Non-Preemptive Scheduling (FCFS)
// By Sheikh Wasimuddin
#include <stdio.h>
int main() {
    int n, i;
    int at[100], bt[100], wt[100], tat[100], ct[100];
    float total_wt = 0, total_tat = 0;

    printf("Enter  Total Number Of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter The Arrival Time for P%d: ", i + 1);
        scanf("%d", &at[i]);
        printf("Enter The  Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort
    for (i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                int t;
                t = at[j];
                at[j] = at[j + 1];
                at[j + 1] = t;
                t = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = t;
            }
        }
    }

    ct[0] = at[0] + bt[0];
    tat[0] = ct[0] - at[0];
```

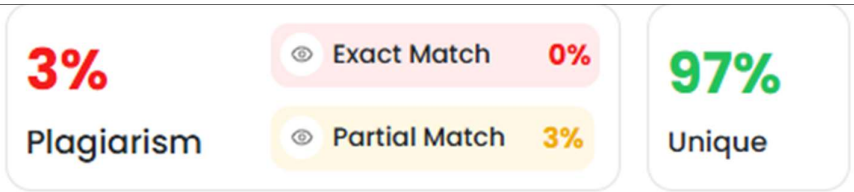
```
Enter  Total Number Of processes: 4
Enter The Arrival Time for P1: 0
Enter The  Burst Time for P1: 5
Enter The Arrival Time for P2: 1
Enter The  Burst Time for P2: 3
Enter The Arrival Time for P3: 2
Enter The  Burst Time for P3: 8
Enter The Arrival Time for P4: 3
Enter The  Burst Time for P4: 6

--- FCFS Scheduling ---
Process AT      BT      CT      TAT     WT
P1      0       5       5       5       0
P2      1       3       8       7       4
P3      2       8       16      14      6
P4      3       6       22      19      13

 The Average WT = 5.75 ms
 The Average TAT = 11.25 ms


...Program finished with exit code 0
Press ENTER to exit console.
```

B. Pre-emptive CPU Scheduling Algorithm SRTF

main.c

```c
// Practical_4_OS - Preemptive Scheduling (SRTF)
// By Sheikh Wasimuddin

#include <stdio.h>

int main() {
    int n, i, time = 0, completed = 0;
    int at[100], bt[100], rt[100];
    int wt[100] = {0}, tat[100];
    float total_wt = 0, total_tat = 0;

    printf("Enter Total Number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter The Arrival Time for P%d: ", i + 1);
        scanf("%d", &at[i]);
        printf("Enter The Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    while (completed != n) {
        int shortest = -1, min_time = 1e9;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0 && rt[i] < min_time) {
                min_time = rt[i];
                shortest = i;
            }
        }
        if (shortest == -1) {
            time++;
            continue;
        }
        rt[shortest]--;
```

```
Enter Total Number of processes: 4
Enter The Arrival Time for P1: 0
Enter The Burst Time for P1: 8
Enter The Arrival Time for P2: 1
Enter The Burst Time for P2: 4
Enter The Arrival Time for P3: 2
Enter The Burst Time for P3: 9
Enter The Arrival Time for P4: 3
Enter The Burst Time for P4: 5

--- SRTF Scheduling ---
Process AT      BT      WT      TAT
P1      0       8       9       17
P2      1       4       0       4
P3      2       9       15      24
P4      3       5       2       7

 The Average WT = 6.50 ms
 The Average TAT = 13.00 ms


...Program finished with exit code 0
Press ENTER to exit console.
```

Output Analysis	**A. Non-Preemptive CPU Scheduling.** Execution order: **P1 → P2 → P3 → P4** (since FCFS is based on arrival). Average Waiting Time = 5.75 ms, Average Turnaround Time = 11.25 ms Observation**:** Early processes finish quickly, but later processes wait longer. **B. Pre-emptive CPU Scheduling Algorithm** Execution order (Gantt chart): **P1 → P2 → P4 → P1 → P3** Average Waiting Time = 6.5 ms, Average Turnaround Time = 13.0 ms Observation**:** Short processes are scheduled earlier (preemption), which lowers waiting time compared to FCFS.
Conclusion	In this experiment, we implemented FCFS and SRTF scheduling algorithms. FCFS is simple but can increase waiting time for later processes, while SRTF improves average waiting and turnaround times by prioritizing shorter jobs. The choice of algorithm depends on system needs for fairness or efficiency.
Plag Report (Similarity index < 12%)	
Date	18/08/2025